FormulaSync: A Coordination of Shared State and Formulas in Collaborative Spreadsheets

Korhan Deniz AKIN

Contents

1 General Information **2**

2 Problem Definition **2**

3 Related Work **2**
3.1 Semantics for CRDT-based Collaborative Spreadsheets 2
3.2 AegisSheet: A Compositional CRDT for Collaborative Spreadsheets 2
3.3 Conflict-free Replicated Spread Sheets 3
3.4 Luckysheet 4
3.5 Univer 5
3.6 TinyCld Calc 5
3.7 Framacalc 5

4 Representing Spreadsheet Data in Yjs **5**
4.1 Overview 5
4.2 YColumns & YRows (SpreadSheet.tsx) 5
4.3 YMap (Cell.tsx) 6
4.4 YColKeep & YRowKeep (SpreadSheet.tsx) 6

5 Formula Support **6**
5.1 Information 6
5.2 Implementation Details 7
5.2.1 Formula Parser 7
5.2.2 Spreadsheet Store 8
5.2.3 SpreadSheet Component 8
5.2.4 Cell Component 8
5.3 Circular Reference Handling 9
5.4 Benchmarking 9

6 CSV Support **9**

1 General Information

In this research, the aim is to build a collaborative spreadsheet that supports formula parsing like Microsoft Excel. React.js and TypeScript are used to build the application. Typescript is chosen instead of Javascript because it provides strict typing. Synchronization across multiple clients was achieved by Yjs which is a JavaScript framework that uses Conflict-free Replicated Data Types (CRDTs) with an abstraction of shared data Jahns, 2026. Zustand is used for global React state management. Real-time updates are transmitted to the connected peers using a WebSocket layer which is implemented by y-websocket.

2 Problem Definition

3 Related Work

3.1 Semantics for CRDT-based Collaborative Spreadsheets

Our research builds upon the work of Yanakieva et al. (2023). In their work, they compared several data models on implementing collaborative spreadsheets and proposed an implementation which supports conflict resolution for offline operations using a combination of existing replicated data types provided by Yjs. They proposed a new data model called "Fixed Elements (Remove-keep)" to prevent data loss when a user modifies a cell and, at the same time, another user deletes the corresponding column or row. They did this by using two y-arrays (to keep track of the orders of cell IDs), a y-map (to associate cell IDs to their contents) and two "keep" y-maps for columns and rows. The idea is to store the "keep" IDs in these maps when there is a "positive update" on a cell. If there is a removal operation concurrent to the positive update, the "keep" IDs are used to undo the removal operation. Y.UndoManager was used to implement the undo operation. This implementation is used in our research as well. Update-wins CRDT for cell content and formula support is also implemented in our research which was not present in the original work.

3.2 AegisSheet: A Compositional CRDT for Collaborative Spreadsheets

Another relevant work is AegisSheet, which uses a composition of CRDTs for collaborative spreadsheets. It was implemented by Pfeil et al. (2026). Their implementation involves conflict resolution for:

- Concurrent cell edit and column/row deletion as similar to the approach used in the work of Yanakieva et al. (2023),
- concurrent cell edits by using Multi-Value CRDTs (allowing user to select the desired value),

- concurrent cell edit and column move,
- concurrent range creation and column deletion.

In their implementation, rows and columns were stored using identifiers and cell content was stored in a map with keys following orders of row/column ID pairs. In this form, it carries similarities to the previous study as well as to our own study.

Their implementation also supports moving columns/rows and an undo feature (which are not implemented in our research). As demonstrated by Kleppmann (2020), standard list CRDTs encounter duplicate elements when moving elements concurrently because move operation requires to delete an element from one position and insert it into another. Therefore, in order to handle this issue, they created a new CRDT called "ReplicatedUniqueList".

They didn't address formula support in their implementation.

3.3 Conflict-free Replicated Spread Sheets

Sypytkowski (2023) proposed an implementation of a collaborative spreadsheet named as "Conflict-free Replicated Spread Sheets".

In his implementation, in comparison to our approach, he didn't use Yjs framework. Instead, he implemented the spreadsheet using standard arrays and LSeq data structure.

He stored spreadsheet data in a single table. This table was constructed as an array of rows, and each row was an array of cells. Each cell carries last-writer-wins semantics. To do this, there is companion 'versions' table containing timestamps for each cell.

In order to keep track of the order of rows and columns, he used LSeq data structure because using a standard array structure does not provide a way to maintain the order of elements in a collaborative environment. For example, with a standard array, if two users concurrently insert elements at the same index, the order of these elements will be lost. The idea behind of LSeq is "Fractional Indexing". Fractional Indexing is an approach which assigns IDs to each of its elements. When a new element is inserted between two elements, the real-number averages of the two elements should be calculated for the new element's ID. With this way, eventual consistency is guaranteed in every replica and there is no need to shift remaining elements. In the implementation of Sypytkowski (2023), these IDs are a combination of sequence of bytes (e.g. byte[]) and the ID of the replica. These sequences are used to order elements with lexical ordering. When a new element is inserted, it is assigned a new ID that is lexically between the IDs of its neighbors. For example, if there are 2 elements with IDs 'a' and 'b', a new element would be assigned an ID that is lexically between 'a' and 'b' (e.g. 'ab').

His implementation consists of 4 main operations:

- Insertion of columns/rows. Each insertion operation assigns a timestamp to each cell which is called as 'version timestamp'.

- Update: What is interesting about this operation is that it expands the table if the user tries to update a window that is out of the current table's bounds. This approach prevents the loss of updates; otherwise, if the table does not expand to apply updates, and if a table updated by one user is larger than the one belong to the other user, updates falling outside the dimensions of the smaller table would be lost. Each update operation assigns a timestamp to each cell which is called as 'version timestamp'.

- Delete: Each delete operation has a timestamp as well. Before any deletion of a column/row, all cells on to be deleted row/column are iterated and the 'version timestamp' of a cell is compared with the deletion timestamp. If the version timestamp is greater than the deletion timestamp, it means there is a concurrent update operation of a cell. In this case, deletion is not performed. If that's not the case the cell value is set to null. If the column/row has only null values left, then it is deleted from the table.

- Selection: To select a window of cells. This operation consists of a set of applications such as formula ranges (e.g. SUM (A1:B2)), applying a set of formatting attributes to multiple cells, merging cells and preventing displayed content to be changed unexpectedly when there is a copy/paste operation from another user. As he mentioned, this operation comes with a problem of concurrent row/column addition or deletion. For example, if a user selects a range of cells (e.g. SUM (A1:B2)) and another user deletes a column/row in that range, the range will differ from our initial selection. To address this issue, he proposed a solution that is based on the idea of logical indexing. Two opposite points of the intended selection are represented as logical indexes. These indexes are again implemented with LSeq data structure.

In his blog, Sypytkowski (2023) mentioned some optimizations as well. Such as "Indexing Selections" using R-Trees for selections in order to listen to changes in the selected cells and recalculate formulas accordingly. However, he didn't implement this feature, and later he mentioned that benefits are negligible compared to the brute-force selection of cells.

3.4 Luckysheet

Another open-source web spreadsheet is **luckysheet**. In contrast to the previously mentioned CRDT-based approaches, Luckysheet relies on a client-server architecture rather than CRDTs.

3.5 Univer

3.6 TinyCld Calc

3.7 Framacalc

4 Representing Spreadsheet Data in Yjs

4.1 Overview

Representing spreadsheet in a collaborative environment was quite straightforward. Yjs framework was very useful because it abstracts all the network broadcasting messages, CRDT implementations, merging updates with CRDTs, and even WebSocket connections. Therefore, none of these aspects are needed to be reimplemented. However, using this framework with React's state management was the hardest part because Yjs only solves synchronization between other clients not how the client's screen is updated and how does the data is displayed. In relevant sections this will be explained further.

The implementation uses 2 arrays and a map. These arrays store cell IDs and are crucial to display them on the table with respect to the order. The map is used to store the content of the cells. This means there is no global grid, which might cause bottlenecks during adding/removing columns/rows so, instead of storing the entire table as a matrix, the sheet is stored into separate structures.

4.2 YColumns & YRows (SpreadSheet.tsx)

These are CRDT-based arrays that are responsible for ordering cells in the spreadsheet. Using a map structure is not enough to ensure ordering, because maps are unordered.

They appear in the SpreadSheet.tsx component. In this component, YColumns and YRows are used to transfer cell IDs to the Spreadsheet's React State when this component is mounted. Later, 'columns' and 'rows' states are used in the table to display the content. They show how the spreadsheet synchronizes the Yjs shared column data with the local React state. Because Yjs data structures live outside the React state, changes in Yjs do not trigger a re-render in the user interface (as mentioned before). An observer function inside 'useEffect' bridges this gap. When a user adds or removes a column, the observer function receives an event:

- Insertions: When new column IDs are inserted, their target index is found using 'findYArrayElement' function. The new column is inserted into the local 'columns' array.
- Deletions: Deleted column IDs are collected from into a data structure. Then the local React state is then updated by filtering out these deleted columns.

4.3 YMap (Cell.tsx)

To keep values of each cell, YMap structure was used. It's a CRDT-based key-value (Yjs provided abstraction for CRDTs as mentioned) pair where IDs are defined as:
`let cellId: string = ‘${col.id},${row.id}‘;`
and contents are just texts. To use the YMap effectively, for every Cell on the spreadsheet, an observer is defined. This observer is responsible for keeping track of the remote updates of the Cell's own content. This comes with a performance cost, but the benefits are not negligible and was the most convenient approach so far rather than using a global grid.

First, with this approach, storing Cell components in a global grid was not needed. If there was a global grid, adding or removing rows/columns would be costly because of shifting all the remaining elements.

This observer also has an implementation for Update-Wins CRDT. The idea behind it is to store updates that are concurrent to each other (e.g., offline clients performing updates) and merge them by concatenation. That's the reason why YMap entries have arrays as their values. Later appendConcurrentUpdates() is called to do string concatenation (merging concurrent updates).

4.4 YColKeep & YRowKeep (SpreadSheet.tsx)

With the standard Yjs, data was lost when a user modified a cell and, at the same time, another user deleted the corresponding column or row. To solve this problem, this implementation uses the idea of 'keep' arrays which has been proposed by Yanakieva et al., 2023. This structure prevents data loss mentioned above. The solution involves storing a "keep ID" for each cell that is being modified. When there is a positive update on a cell: "keep ID"
`let keepId: RemoveKeepOperationId = ‘c${YDoc.clientID as number}.${1 as number}‘;`
is stored in those arrays. Clients monitor the changes of arrays with observers, and if they are non-empty, undo operations are performed equal to the number of elements in the array structures.

5 Formula Support

5.1 Information

Supporting formulas in a collaborative environment would be interesting for several reasons:

1. Retrieving spreadsheet data for formula calculation. What is the feasible way of doing this without ruining the spreadsheet if there are offline users too?

2. Prevention of circular referencing during calculation and recalculation.

3. Reactivity of the formula cells. When a formula is defined, the corresponding cells are marked. When there is an update on these marked cells, the formula should be recalculated and the result should be displayed in the formula cell.

4. Shifting ranges of cells when a new row or column is added before or above the cell that stores the range of cells, or when such a row or column is deleted.

5. Concurrent row/column addition and deletion between range of cells that are used in a formula.

6. Formula recalculation for every update and its probable bottlenecks. For example, let's say a result is computed for a formula using 10000 cells and a user updates one cell value by incrementing it by one. In this scenario, should the formula be calculated from the very beginning, or is it possible to support this functionality more efficiently? While the current implementation does so by recalculating the formula, implementing more efficient algorithms is possible.

This implementation addresses the first 5 points. The last point is not implemented yet, but it is a possible future work.

5.2 Implementation Details

In this section, relevant parts of functions/components for formula calculation will be explained.

5.2.1 Formula Parser

First, the `'fast-formula-parser'` Lyu, 2020 library is used for formula functionality. With this library, my implementation supports basic arithmetic operations, cell references, and range references. The main implementation for this library is in 'Formula.ts'. This class is responsible for parsing the formula, detecting circular references, and recalculating the formula when there are updates on the marked cells. A pop-up is displayed when a circular reference is detected. For this, SweetAlert2, n.d. library is used.

This class has several attributes, which are: 'formula', 'cellId', 'position', 'formulaResult', 'markedCells', 'operation', 'leftBoundaries', and 'rightBoundaries'. Attributes for left and right boundaries are used to store the leftmost and rightmost cell row and column names in a formula range (e.g. SUM(A1:B2), left boundary: A1, right boundary: B2). They are stored in a priority queue because when a new row or column is added in between the left and right boundaries (even concurrently), the left and right boundaries should be updated by peeking priority queues. For example, if the formula is SUM(A1:B2) and a new column is added between A and B, the new formula should be SUM(A1:C2). To implement a priority queue, the implementation of Aissi (2024) was used.

There are functions in this class which are: 'parser()', which comes from the library itself, and 'parserDriver()' which calls the parser() with 'spreadsheetData' obtained in the Spreadsheet component.

This parserDriver() function returns the formula result and returns the IDs of the cells that have been marked (or registered) while computing the formula result.

5.2.2 Spreadsheet Store

To retrieve the store globally, Zustand, which is a small, fast, and scalable state management solution Poimandres, 2026, is used. It is like Redux but was easier to implement.

Normally, the direction for the data transfer is from the parent to the child. However, it would not have been feasible to transfer all spreadsheet data from the Spreadsheet component to every Cell. The second solution requires spreadsheet data*num of total cells. With Zustand, there is a single spreadsheet used for formula calculations.

'spreadSheetData' is the name of the object that contains all values in each cell and is a matrix of strings obtained from 'SpreadSheetStore.ts': 'returnSpreadSheet()'. As mentioned in section 3.1 this function returns all spreadsheet data (optimization might be possible).

5.2.3 SpreadSheet Component

The SpreadSheet component provides a table by implementing the PrimeReact library's DataTable structure with the Yjs framework. While the table's rows and columns are tracked via Yjs arrays (yRows, yColumns), observers ensure that all additions and deletions are reflected in the local React state. When a new row or column is created, the corresponding cells are automatically created in the shared map, and deletion operations trigger both map cleanup and undo mechanisms. This component also consists of other components, which are headers (ColumnHeader), cells (Cell), and buttons.

handleFormula()

5.2.4 Cell Component

handleFocusOut() Formula cells register candidate cells in the 'markedCells[]' array when the formula is defined to compute the result. Similarly, candidate cells store IDs of the formula cells in the 'formulaReferenceCells[]' array. This is an array because there might be multiple formula cells registering the same cell. These 2 arrays are kept in every Cell component and are defined in 'CellType'. Arrays are optional, denoted with "?", so it is not a problem if they are undefined.

After these operations, the 'handleFocusOut()' function in the Cell component is responsible for the formula recalculation.

5.3 Circular Reference Handling

A circular reference occurs when a formula refers directly back to its own. This creates an endless calculation loop. For example, if cell A1 contains the formula SUM(A1+B1), then A1 references itself.

The implementation does not produce an endless loop in the definition phase because formula recalculation does not use observers. However, when there are updates to cells marked by formulas, a circular reference becomes apparent. Rather, it is being done via the 'handleFocusOut()' function in Cell components. Therefore, a formula is calculated via 'handleFocusOut()' calls. Even though there are no calculation infinite loops, when a user wants to increase a cell value by one (if the cell is marked and the formula is SUM), as in a circular reference case, formula cells register themselves, the existing result from the previous step, so it does not increase by one rather formula's previous result + 1. This produces a result that the user does not intend.

To solve this, Microsoft Excel's approach is used. When it detects a circular reference, the formula value is set to 0. In my implementation, I have a very basic approach that can be seen in Figure **??**. This approach might be improved in terms of time complexity via Depth-First Search.

5.4 Benchmarking

6 CSV Support

References

Aissi, M. H. (2024, July). Priority queue in TypeScript implementation, updating priority of heap elements, removing elements while preserving the heap properties [Medium. Accessed: 2026-10-24].

Jahns, K. (2026). Yjs: Shared data types for building collaborative software [Accessed: 2026-08-23].

Kleppmann, M. (2020). Moving elements in list CRDTs. *Proceedings of the 7th Workshop on Principles and Practice of Consistency for Distributed Data*, 1–6. https://doi.org/10.1145/3380787.3393677

Lyu, L. (2020). Fast-formula-parser [Accessed: 2026-07-24].

Pfeil, F., Scandurra, D., & Haas, J. (2026). AegisSheet: A compositional CRDT for collaborative spreadsheets. *Proceedings of the 13th International Workshop on Principles and Practice of Consistency for Distributed Data*, 40–46. https://doi.org/10.1145/3806077.3806695

Poimandres. (2026). Introduction - Zustand documentation [Accessed: 2026-07-25].

SweetAlert2. (n.d.). SweetAlert2 - beautiful, accessible JavaScript popup boxes [Accessed: 2026-10-24].

Sypytkowski, B. (2023, October). *Conflict-free replicated spread sheets*. Retrieved September 29, 2026, from https://www.bartoszsypytkowski.com/crdt-tables/

Yanakieva, E., Bird, P., & Bieniusa, A. (2023). A study of semantics for CRDT-based collaborative spreadsheets. *Proceedings of the 10th Workshop on Principles and Practice of Consistency for Distributed Data*, 37–43. https://doi.org/10.1145/3578358.3591324